Lec-34

06/04/25 2:10 PM

Next Greater Element

Given an array of integers, for each element in the array, find the **next greater element** (NGE) in the array. The next greater element for an element x is the first element y to the right of x in the array such that $y > x$. If no such element exists, the NGE for that element is **-1**.

[4 5 2 10]

5 10 10 -1

O(n²)

O(n)

[So, 5, 3 2, 8, 7, 9, 11 3]

-1 8 8 8 9 9 11 -1 -1

[So, 5, 3 2, 8, 7, 9, 11 3]

8 8 8 9 11

[So, 5, 3 2, 8, 7, 9, 11 3]

-1	8	8	8	9	9	11	11	-1
0	1	2	3	4	5	6	7	8

for(i=0 i<arr.len i++)

while(!st.isempty() && arr[i] > arr[st.peek()]

ans[st.pop()] = arr[i]

st.push(i)

while(!st.isempty()) ans[st.top()] = -1

Stock Span

The stock span problem is a financial problem where we have a series of N daily price quotes for a stock and we need to calculate span of stock's price for all N days. You are given an array of length N, where i^{th} element of array denotes the price of a stock on i^{th}. Find the span of stock's price on i^{th} day, for every $1<=i<=N$.

A span of a stock's price on a given day, i, is the maximum number of consecutive days before the $(i+1)^{th}$ day, for which stock's price on these days is less than or equal to that on the ith day.

Input format

```
First line contains integer N denoting size of the array.

Next line contains N space separated integers denoting the elements of the array.
```

Input

```
5
30
35
40
38
35
```

Output

```
1 2 3 1 1 END
```

[30, 35, 40, 38, 35]

1 2 3 1 1

[So, 5, 3 2, 8, 7, 9, 11 3]

1 1 1 4 1 6 7 1

[So, 5, 3 2, 8, 7, 9, 11 3]

1 1 4 1 6 7

for(i=0 i<arr.len i++)

while(!st.isempty() && arr[i] > arr[st.peek()])

st.pop()

if(st.isempty()) ans[i] = i+1

else ans[i] = i - st.peek()

st.push(i)

i = 12

j = 9

i - st.peek()

4 - 0 = 4

2-1=1

3-2=1

6-0=

Given an array of integers `heights` representing the histogram's bar height where the width of each bar is `1`, return *the area of the largest rectangle in the histogram.*

Example 1:

Input: heights = [2,1,5,6,2,3]
Output: 10
Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 units.

[2, 3 5 4, 6, 1, 7]

2×5=10

3×4=12

5×1=5

4×3=12

6×1=6

1×7=7

7×1=7

R-L+1

[2, 3 5 4, 6, 1, 7]

h = arr[i] → 5

h = arr[i] = 6

h = arr[i] = 4

h = arr[i] = 3

h = arr[i]

5×(3-1-1)=5

6×(5-3-1)=6

4×(5-1-1)=12

3×(5-0-1)=2

2×5=10

R=5

[2, 3 5 4, 6, 1, 7]

[2, 3 5 4, 6, 1, 7]

for(i=0 i<arr.len i++)

while(!st.isempty() && arr[i] < arr[st.peek()])

h = st.pop(), h = arr[st.pop]

if(st.isempty()){

ans = max(ans, (h×i))

}

i = st.peek()

ans = max(ans, h×(R-L-1))

}

st.push(i)

}

8 & 12

[2, 3 5 4, 6, 1, 7, 0]

R = 5

R = N

5(3-1-1)=5

6×(5-3-1)=6

4×(5-1-1)=12

3×(5-0-1)=12

2×5=10

7×(7-5-1)=7

1×7=7

```
public static int Largest Rectangle(int[] arr) {
        Stack<Integer> st = new Stack<>();
        int ans = 0;
        for (int i = 0; i < arr.length; i++) {
            while (!st.isEmpty() && arr[i] < arr[st.peek()]) {
                int r = i;
                int h = arr[st.pop()];
                if (st.isEmpty()) {
                    ans = Math.max(ans, h * r);
                } else {
                    int l = st.peek();
                    ans = Math.max(ans, h * (r - l - 1));
                }
            }
            st.push(i);
        }
    }
```

```
int r = arr.length;
        while (!st.isEmpty()) {
            int h = arr[st.pop()];
            if (st.isEmpty()) {
                ans = Math.max(ans, h * r);
            } else {
                int l = st.peek();
                ans = Math.max(ans, h * (r - l - 1));
            }
        }
```

1 0 1 1 0 0

6 ×

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

1 0 0 1 1 0

2 1 1 1 2 1

0 0 2 1 1 0

4 0 3 0 0